Unit 3: Computation of Data Cubes, tuning and testing of Data Warehouse

Lecturer : Mukesh Prasad Chaudhary

Outline

- Measures of Data Cube
- Computation of Data Cubes
- OLAP Operation
- OLAP queries
- Data Warehouse back end tools
- Tuning and Testing of Data Warehouse

Measures of Data Cube

- ***"How are measures computed?"***

A data cube *measure* is a numerical function that can be evaluated at each point in the data cube space.

A measure value is computed for a given point by aggregating the data corresponding to the respective dimension-value pairs defining the given point.

Measures can be organized into three categories (i.e., distributive, algebraic, holistic), based on the kind of aggregate functions used.

Distributive Measure

- A measure is *distributive* if it is obtained by applying a distributive aggregate function.
- An aggregate function is *distributive* if it can be computed in a distributed manner.
- Example: count(), sum(), min(), max().

Algebraic Measure

- A measure is *algebraic* if it is obtained by applying an algebraic aggregate function.
- An aggregate function is *algebraic* if it can be computed by an algebraic function.
- Example: avg(), min_N(), standard_deviation().

Holistic Measure

- A measure is *holistic* if it is obtained by applying a holistic aggregate function.
- An aggregate function is *holistic* if there is no constant bound on the storage size needed to describe a sub-aggregate.
- Example: median(), mode(), rank().

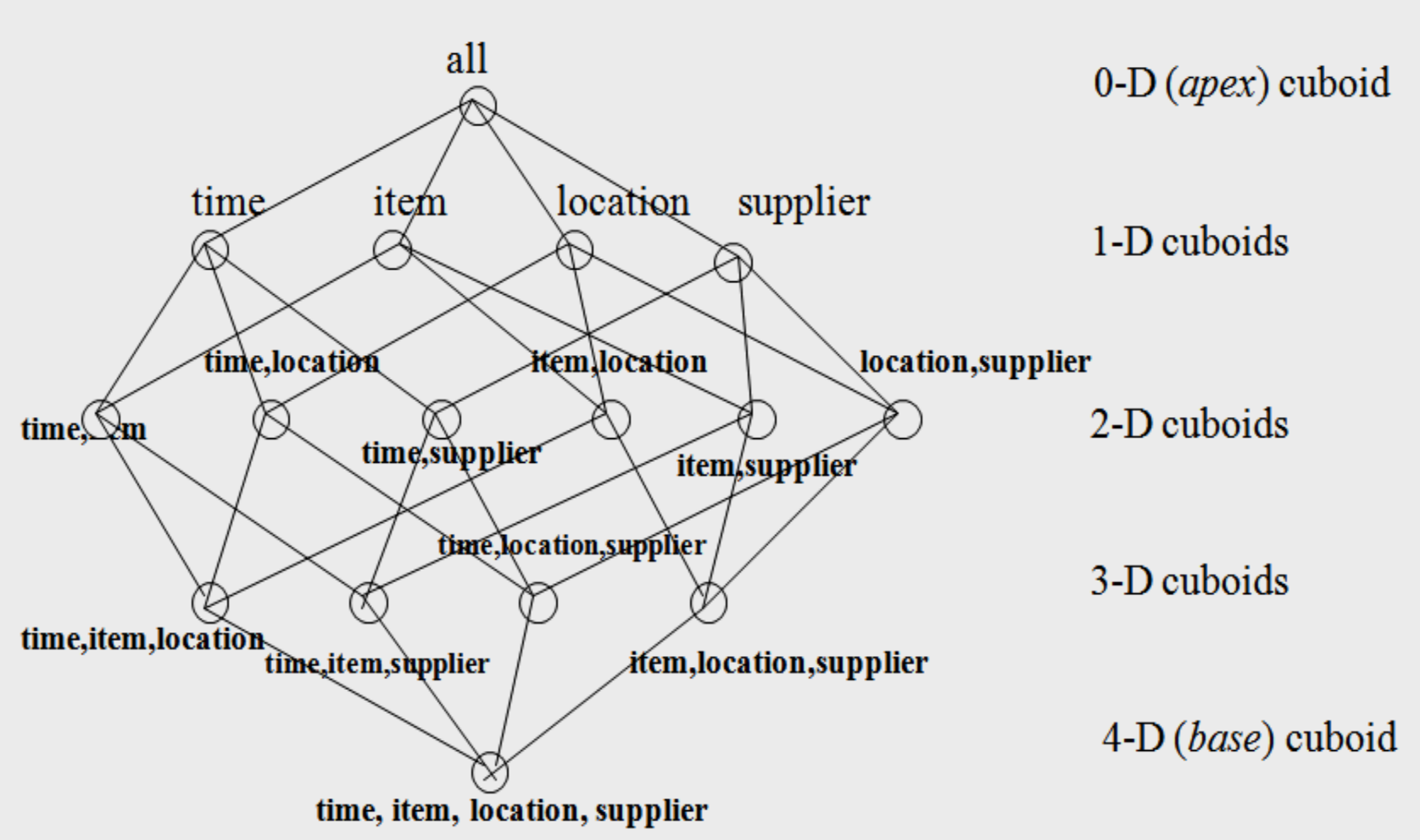
Efficient Data Cube Computation

Data cube can be viewed as a lattice of cuboids

1. The bottom-most cuboid is the base cuboid
2. The top-most cuboid (apex) contains only one cell
3. How many cuboids in an n-dimensional cube with L levels?

$$T = \prod_{i=1}^{n} (L_i + 1)$$

Cube: A Lattice of Cuboids



Efficient Data Cube Computation

- Data cube computation is an essential task in data warehouse implementation.
- The pre-computation of all or part of a data cube can greatly reduce the response time and enhance the performance of on-line analytical processing.
- However, such computation is challenging because it may require substantial computational time and storage space.

Materialization

- Materialized views are query results that have been stored in advance so long-running calculations are not necessary when you actually execute your SQL statements.
- From a physical design point of view, materialized views resemble tables or partitioned tables and behave like indexes in that they are used transparently and improve performance.

CUBE MATERIALIZATION

- Full Cube
- Iceberg Cube
- Closed Cube
- Shell Cube

FULL CUBE

- **Full Cube**:-all the cells of all of the cuboids for a given data cube. Thus, pre computation of the full cube can require huge and often excessive amounts of memory .
- **Full materialization** refers to the computation of all of the cuboids in a data cube lattice.

ICEBERG CUBE

- **Iceberg Cube:-**partially materialized cubes are known as iceberg cubes. The minimum threshold is called the minimum support threshold, or minimum support(min sup)
- **Partial materialization** refers to the selective computation of a subset of the cuboid cells in the lattice. Iceberg cubes and shell fragments are examples of partial materialization.
- **An iceberg cube** is a data cube that stores only those cube cells whose aggregate value (e.g., count)is above some minimum support threshold.

ICEBERG CUBE(EXAMPLE)

- An iceberg cube can be specified with an SQL query , as shown in the following example.

```
compute  cube sales iceberg as
select month, city ,  customer group, count(*)
from salesInfo
cube by   month, city ,  customer group
having  count(*) >= min sup
```

CLOSED CUBE

- T o systematically compress a data cube, we need closed cell.
- **A closed cube** is a data cube consisting of only closed cells.
- A cell, c, is a closed cell if there exists no cell, d, such that d is a specialization (descendant) of cell c (i.e., where d is obtained by replacing a in c with a non- value), and d has the same measure value as c.

CLOSED CUBE

- For example, the three cells derived above are the three closed cells of the data cube for the data set: {(a1, a2, a3, - - - -, a100) : 10, (a1, a2, b3, - - - -, b100) : 10}.They form the lattice of a closed cube as shown in Figure. Other nonclosed cells can be derived from their corresponding closed cells in this lattice. For example,“(a1, * ,*, - - - - , *) : 20” can be derived from “(a1, a2, - - - -,*) : 20” because the former is a generalized nonclosed cell of the latter . Similarly , we have “(a1, a2, b3, *,- - - -,*) : 10” .

CLOSED CELL

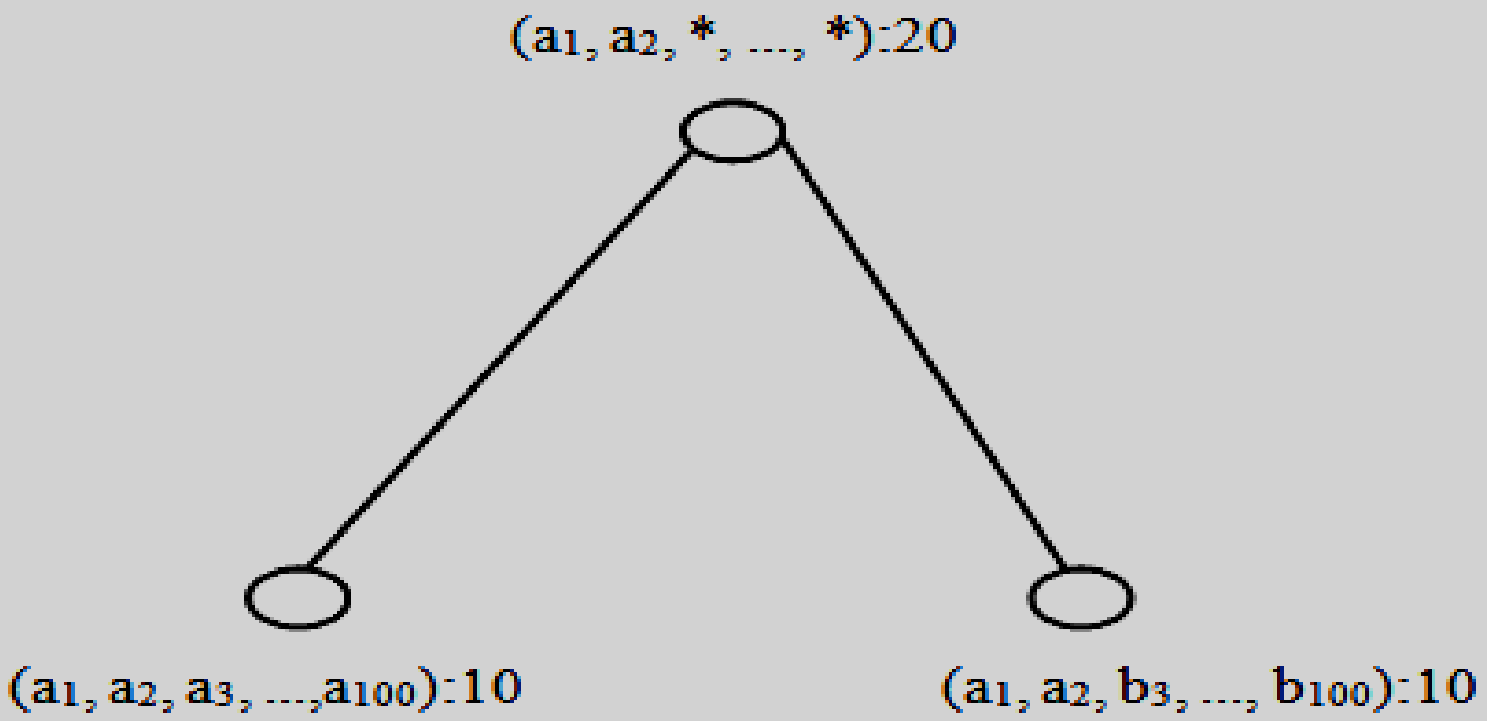


Fig: Three closed cells forming the lattice of a closed cube.

SHELL CUBE

- Another strategy for **partial materialization** is to pre compute only the cuboids involving a small number of dimensions, These cuboids form a cube shell for the corresponding data cube
- For **shell fragments** of a data cube, only some cuboids involving a small number of dimensions are computed. Queries on additional combinations of the dimensions can be computed on the fly .

OPTIMIZATION TECHNIQUES FOR EFFICIENTCOMPUTATION OF DATA CUBES

- **Following are general optimization techniques for the efficient computation of data cubes.**

  1. Sorting, hashing, and grouping.

  2. Simultaneous aggregation and caching intermediate results.

  3. Aggregation from the smallest child, when there exist multiple child cuboids.

  4. The Apriori pruning method can be explored to compute iceberg cubes efficiently .

DATA CUBE COMPUTATION METHODS

1. **Multiway array aggregation (**Top-Down**)** for full data computation.
2. **BUC (**Bottom-up computation) :Computing iceberg cubes by exploring ordering and sorting for efficient top-down computation.
3. **Star-Cubing** for integration of top-down and bottom up computation using a star-tree structure.
4. **High-dimensional OLAP** by pre computing only the partitioned shell fragments.

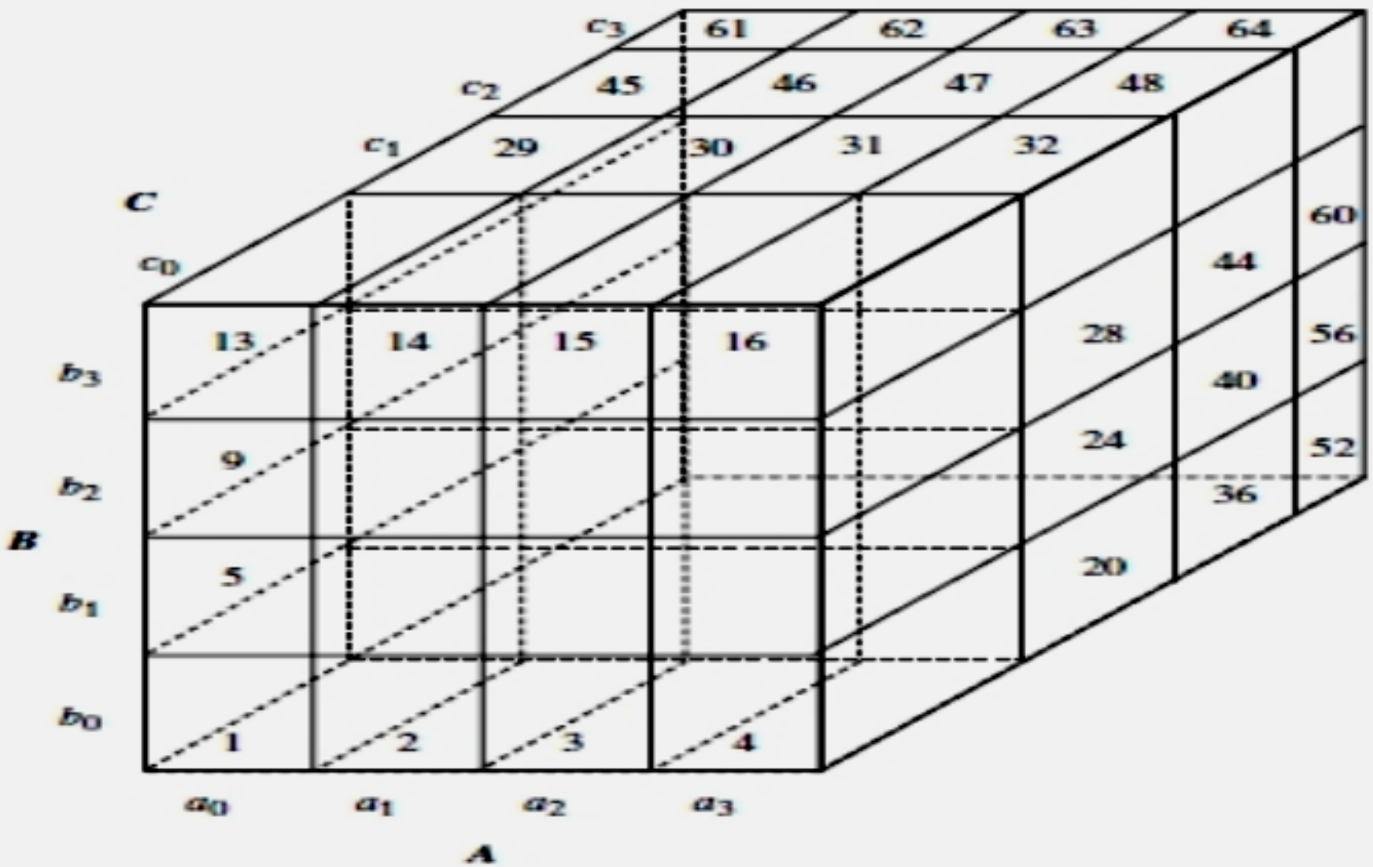
MULTIWAY ARRAY AGGREGATION FOR FULL DATA COMPUTATION

- The Multiway Array Aggregation (or simply MultiWay) method computes a full data cube by using a multidimensional array as its basic data structure.
- It is a typical MOLAP approach that uses direct array addressing, where dimension values are accessed via the position or index of their corresponding array locations.
- Hence, MultiWay cannot perform any value-based reordering as an optimization technique.
- A different approach is developed for the array-based cube construction, as follows:
  - Partition the array into chunks
  - Compute aggregates by visiting cube cells

PARTITION THE ARRAY INTO CHUNKS

- Partition the array into chunks: A chunk is a sub cube that is small enough to fit into the memory available for cube computation.
- Chunking is a method for dividing an n-dimensional array into small n-dimensional chunks, where each chunk is stored as an object on disk.
- The chunks are compressed so as to remove wasted space resulting from empty array cells (i.e., cells that do not contain any valid data, whose cell count is zero). For instance, “chunkID + offset” can be used as a cell addressing mechanism to compress a sparse array structure and when searching for cells within a chunk.
- Such a compression technique is powerful enough to handle sparse cubes, both on disk and in memory .

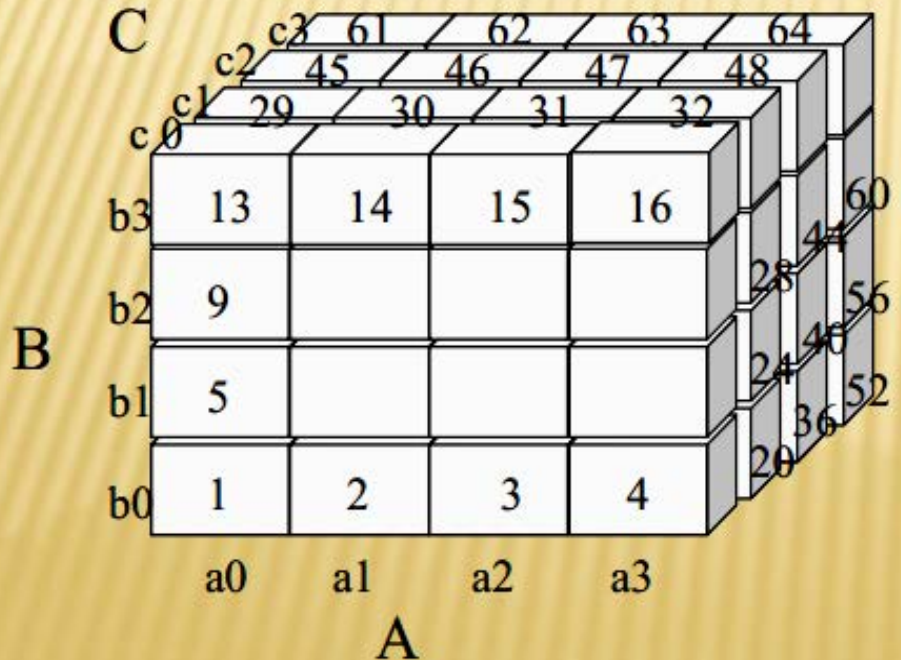
Multi-way Array Aggregation for Cube Computation



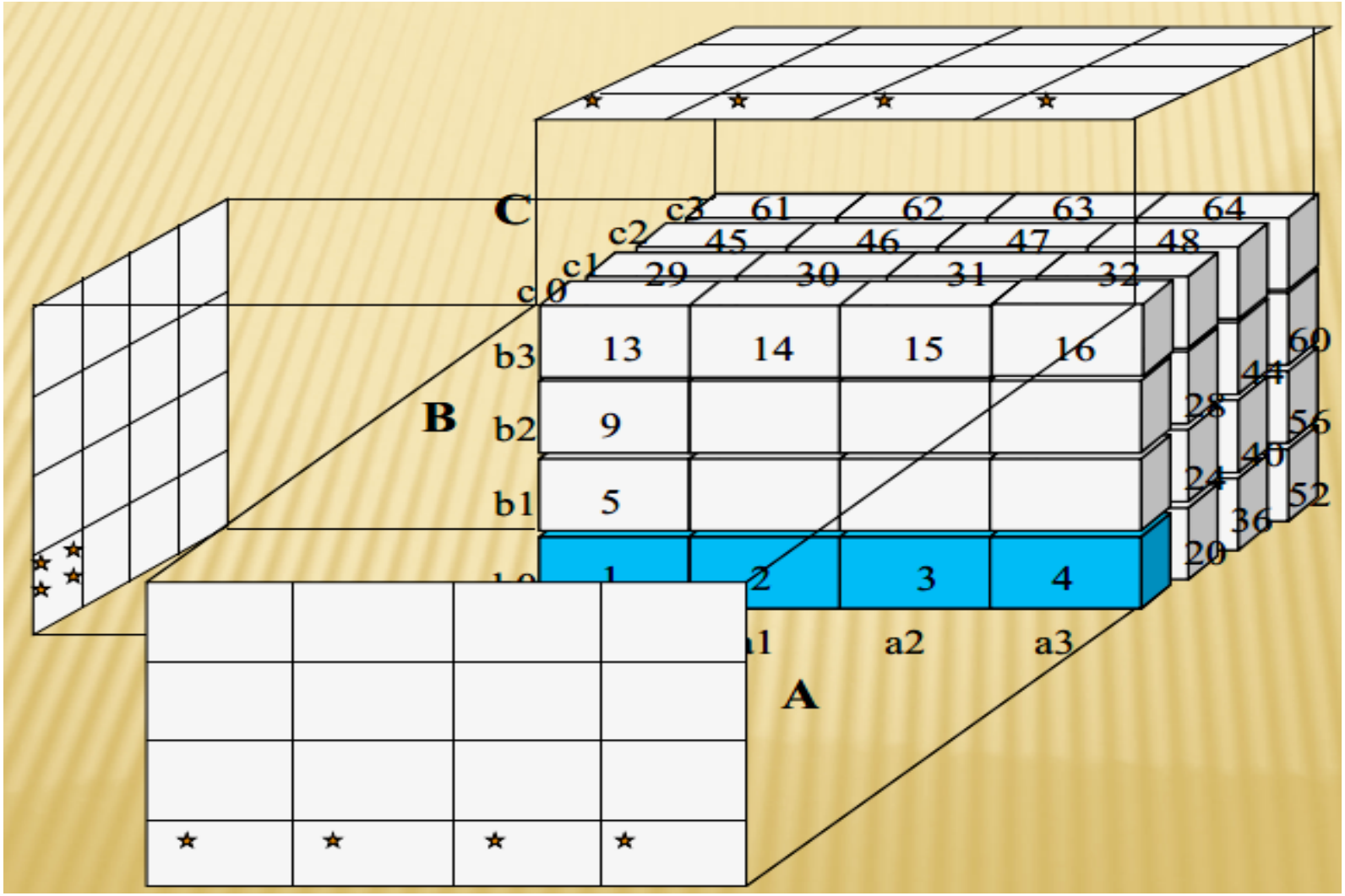
A 3-D array for the dimensions *A*, *B*, and *C*, organized into 64 *chunks*. Each chunk is small enough to fit into the memory available for cube computation.

Multi-way Array Aggregation for Cube Computation

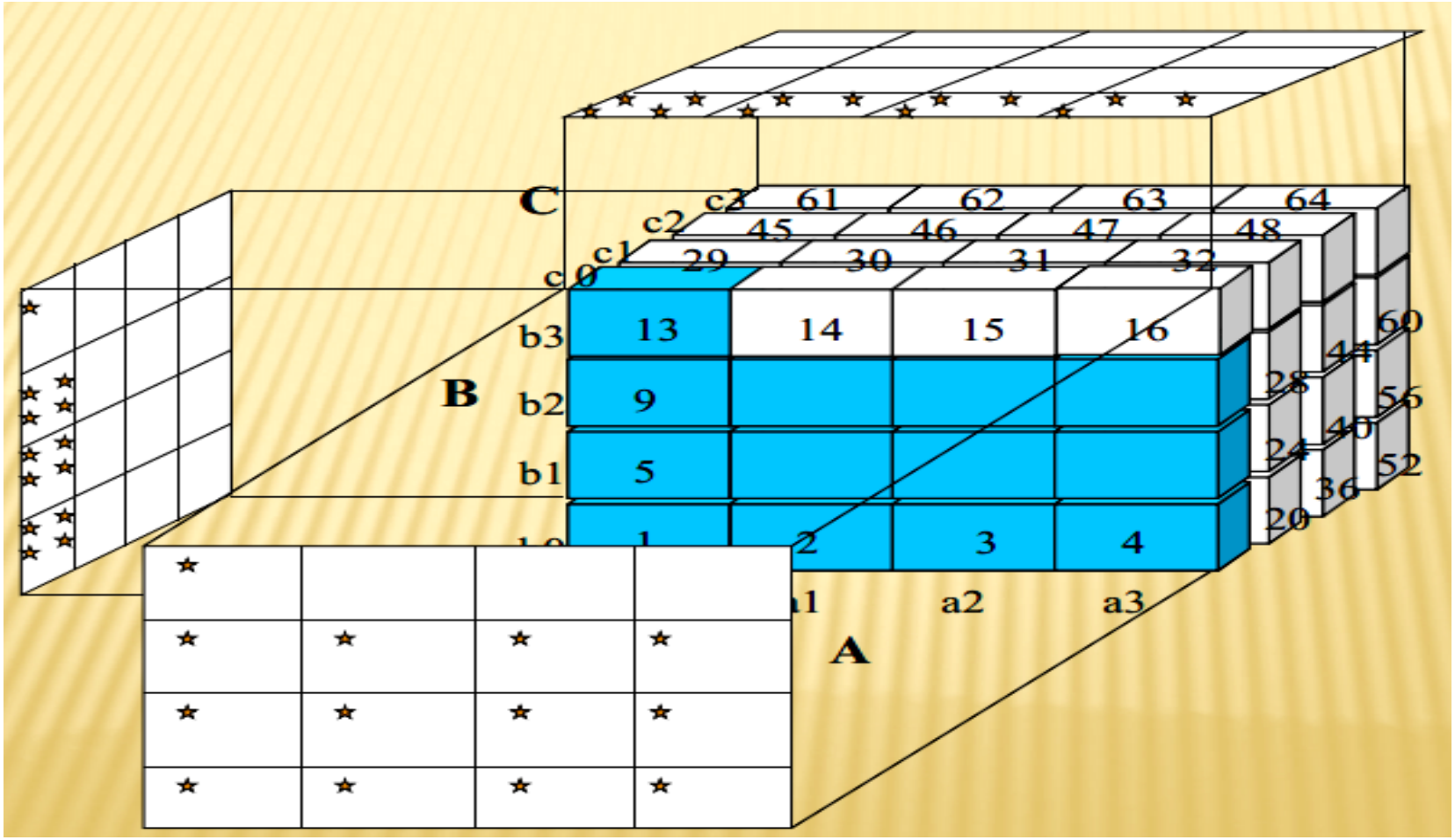
- Partition arrays into chunks (a small subcube which fits in memory).
- Compressed sparse array addressing: (chunk_id, offset)
- Compute aggregates in "multiway" by visiting cube cells in the order which minimizes the # of times to visit each cell, and reduces memory access and storage cost.



Multi-way Array Aggregation for Cube Computation



Multi-way Array Aggregation for Cube Computation



Multi-way Array Aggregation

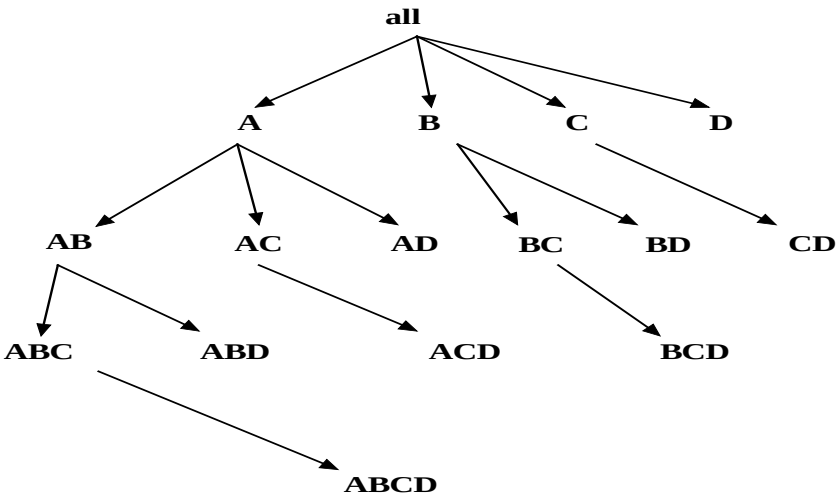
- Method: the planes should be sorted and computed according to their size in ascending order
  - Idea: keep the smallest plane in the main memory, fetch and compute only one chunk at a time for the largest plane
- Limitation of the method: computing well only for a small number of dimensions
  - If there are a large number of dimensions, "top-down" computation and iceberg cube computation methods can be explored

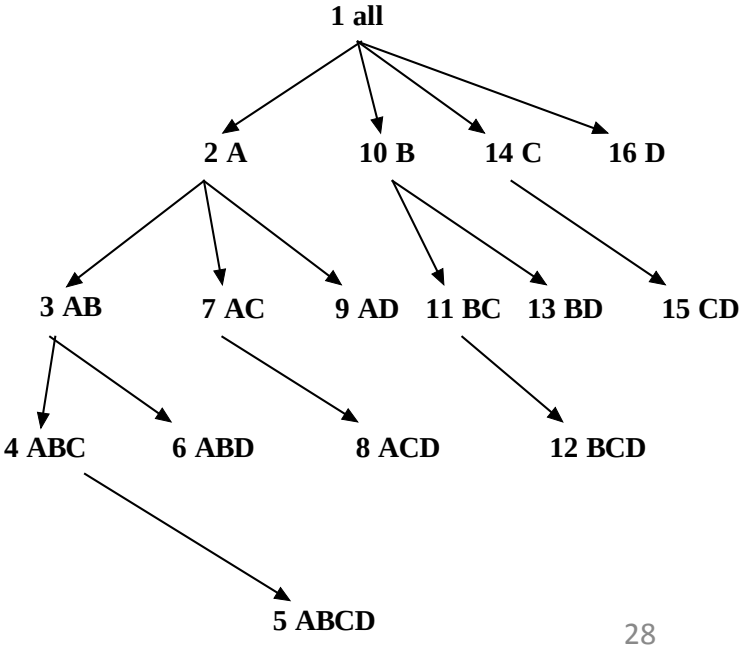
Bottom-Up Computation (BUC)

- Bottom-up cube computation

  (Note: top-down in our view!)

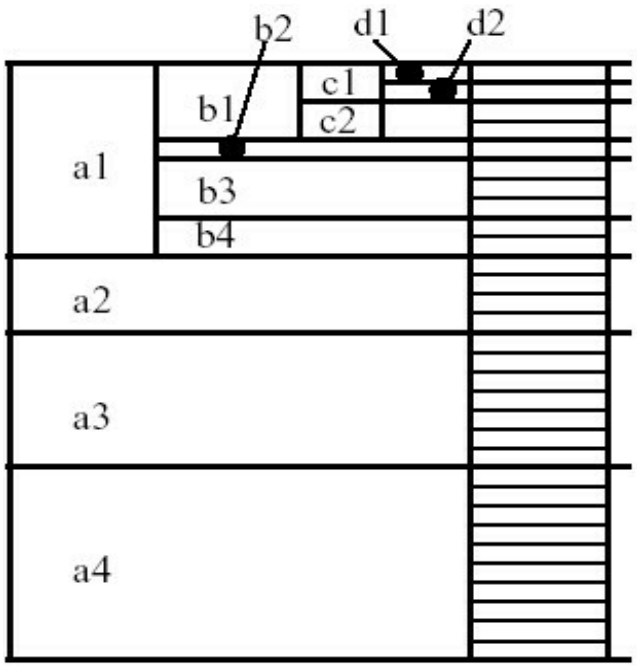
- Divides dimensions into partitions and facilitates iceberg pruning
  - If a partition does not satisfy *min_sup*, its descendants can be pruned
  - If *minsup* = 1 → compute full CUBE!
- No simultaneous aggregation



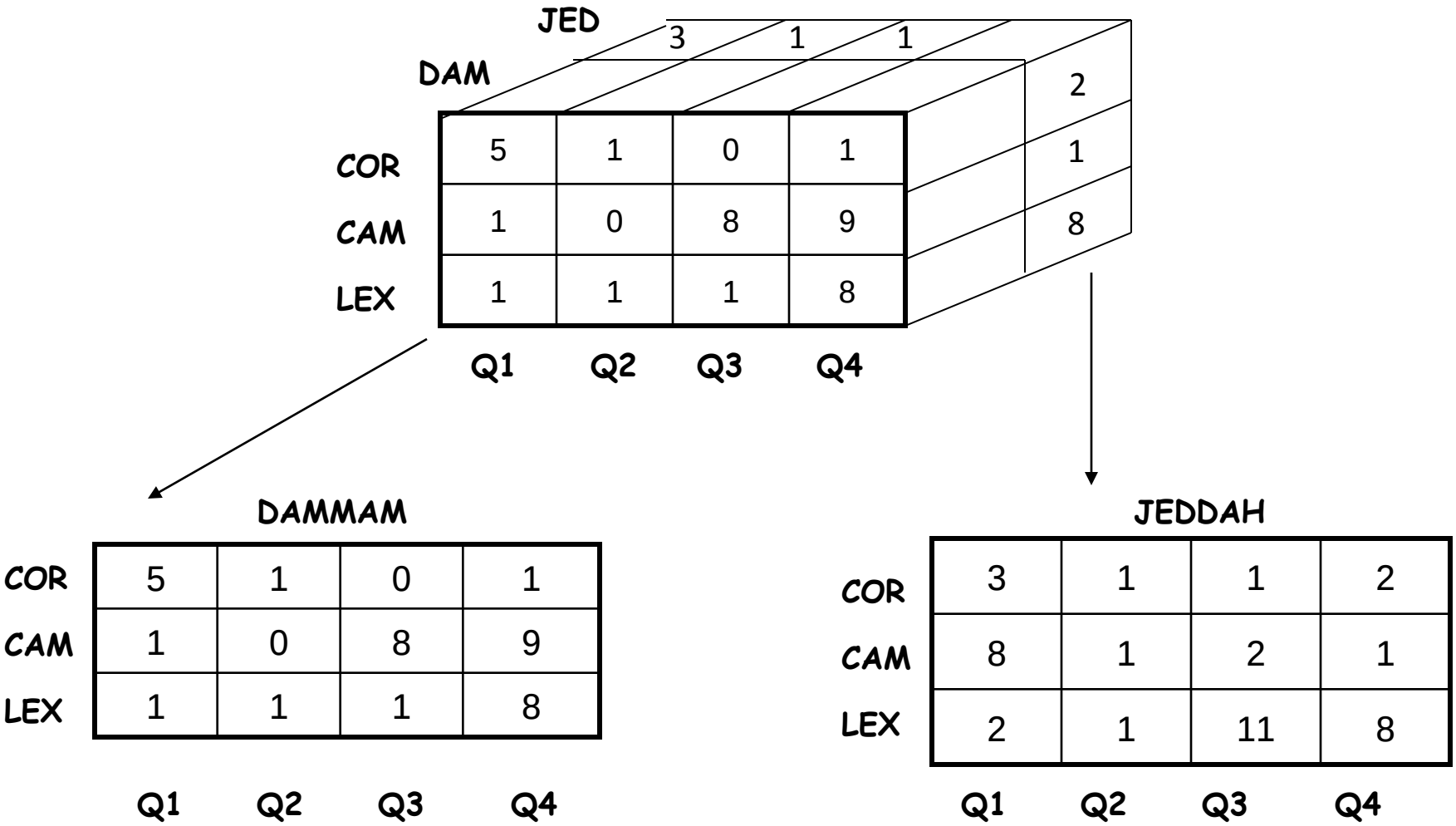


--- BUC: Partitioning

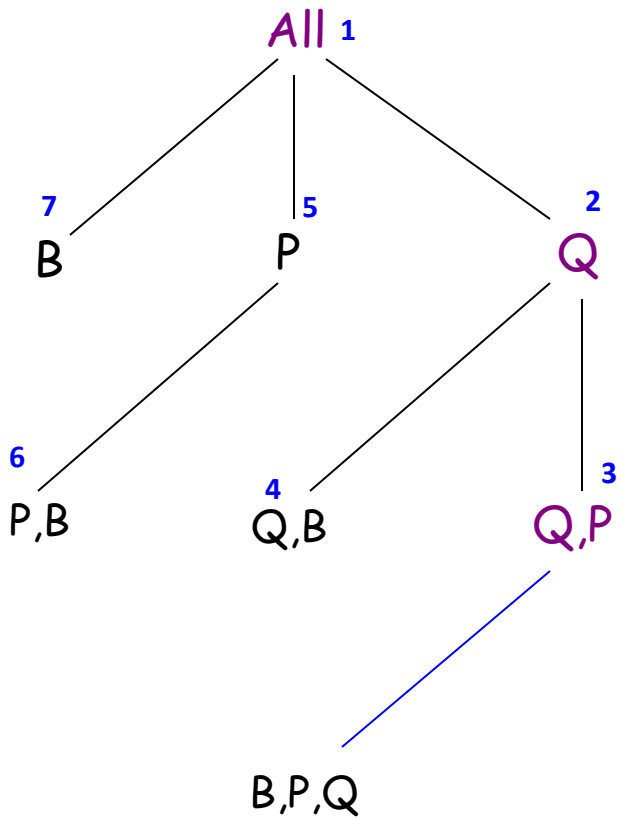
- Usually, entire data set can't fit in main memory
- Sort *distinct* values, partition into blocks that fit
- Continue processing
- Optimizations
  - Partitioning
    - External Sorting, Hashing, Counting Sort
  - Ordering dimensions to encourage pruning
    - Cardinality, Skew, Correlation
  - Collapsing duplicates
    - Can't do holistic aggregates anymore!



--- BUC: Example (Having count(*) > 5) …



… --- BUC: Example (Having count(*) > 5)



All

77

Q

20	5	23	29
Q1	Q2	Q3	Q4

Q,P

COR	8	2	1	3
CAM	9	1	10	10
LEX	3	2	12	16
	Q1	Q2	Q3	Q4

OLAP Operations

- Since OLAP servers are based on multidimensional view of data, we will discuss OLAP operations in multidimensional data.Here is the list of OLAP operations:

❖ Roll-Up (drill-up)

❖ Drill-Down

❖ Slice

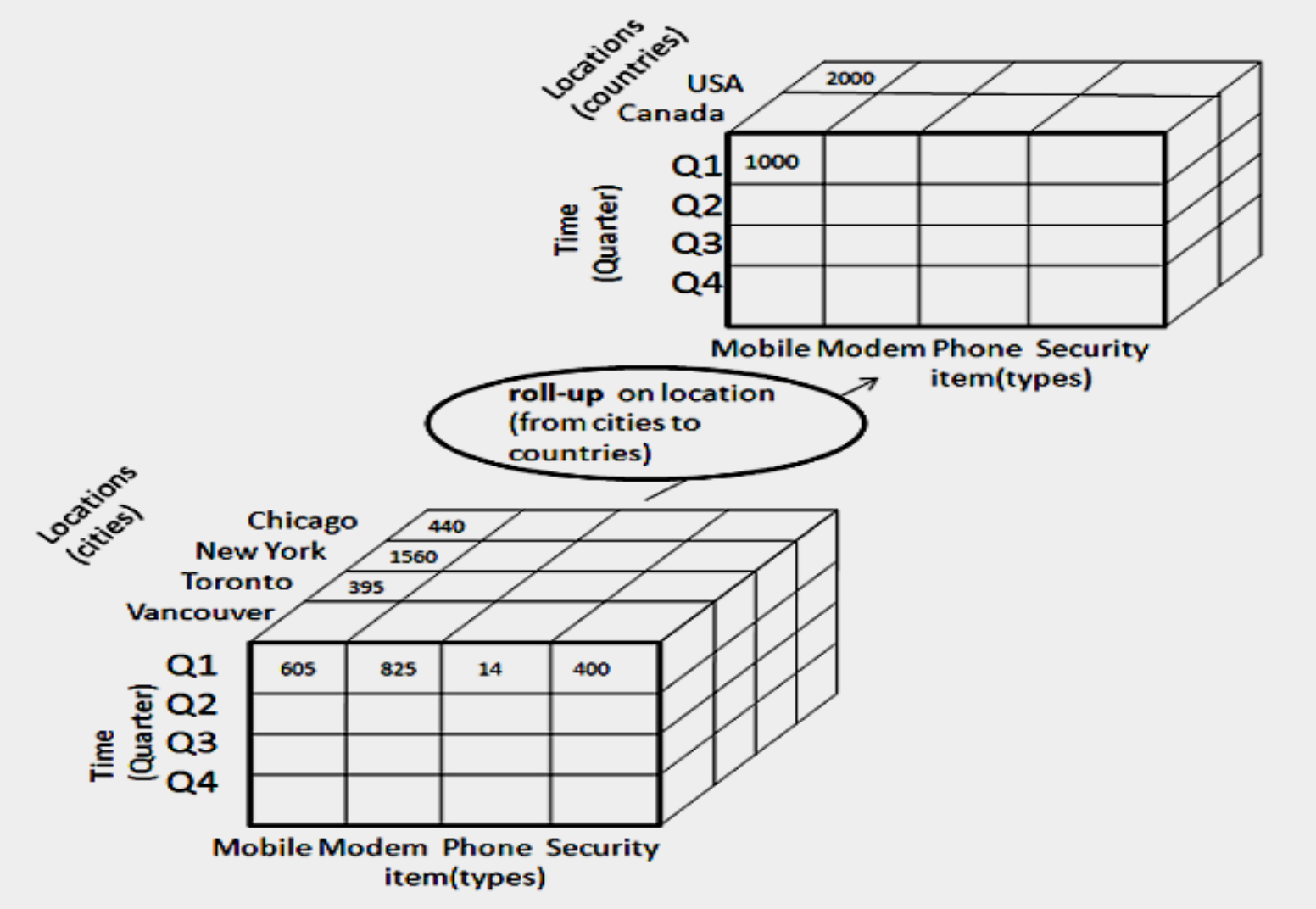
❖ Dice

❖ Pivot(rotate)

Roll-up

- Roll-up performs aggregation on a data cube in any of the following ways:
  - ❖ By climbing up a concept hierarchy for a dimension
  - ❖ By dimension reduction
- Roll-up is performed by climbing up a concept hierarchy for the dimension location.
- Initially the concept hierarchy was "street < city < province < country".
- On rolling up, the data is aggregated by ascending the location hierarchy from the level of city to the level of country.
- The data is grouped into cities rather than countries.
- When roll-up is performed, one or more dimensions from the data cube are removed.

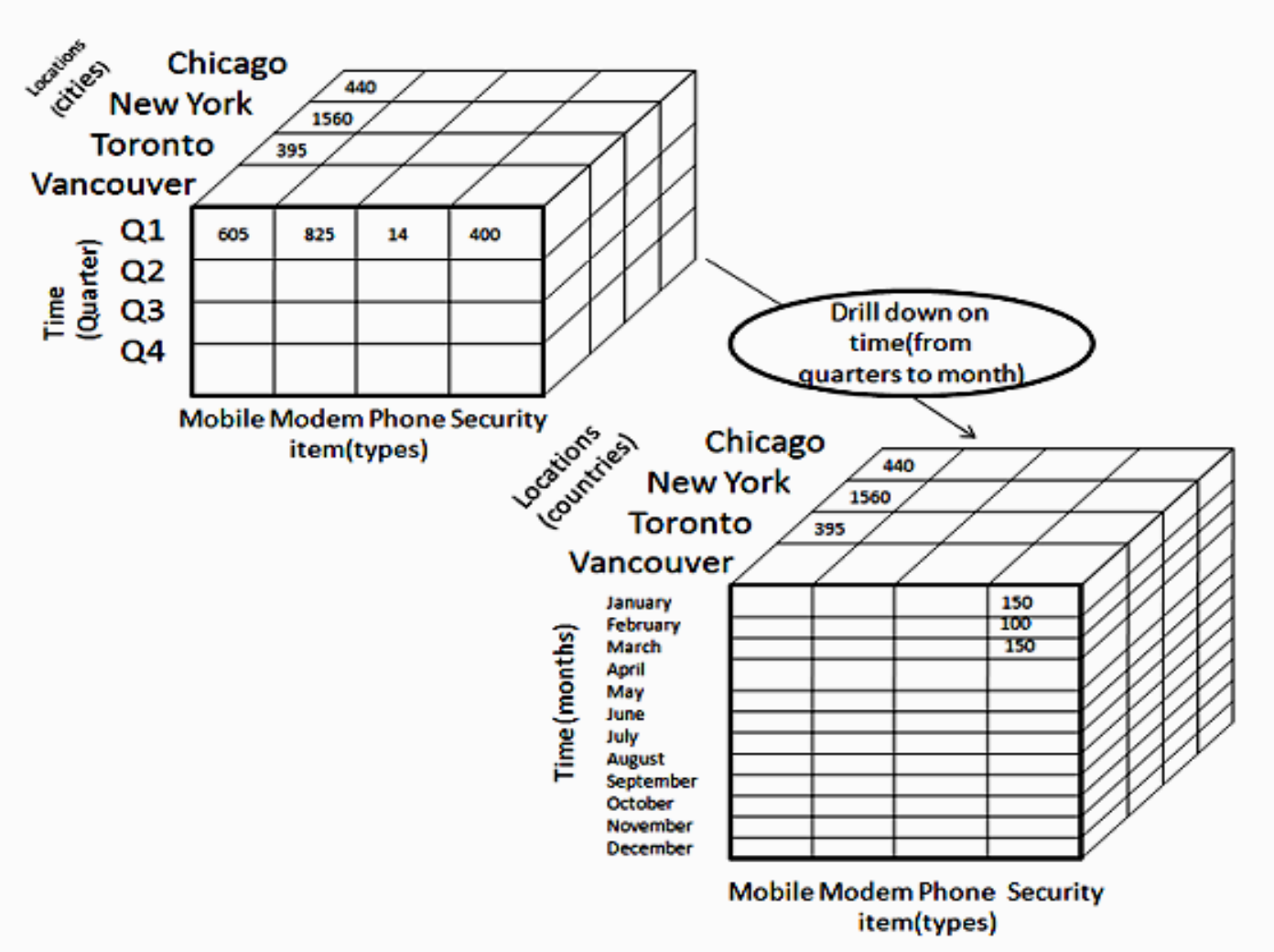
Diagram of how roll-up works



Drill-Down

- Drill-down is the reverse operation of roll-up. It is performed by either of the following ways:
  - ❖ By stepping down a concept hierarchy for a dimension
  - ❖ By introducing a new dimension.
- Drill-down is performed by stepping down a concept hierarchy for the dimension time.
- Initially the concept hierarchy was "day < month < quarter < year."
- On drilling down, the time dimension is descended from the level of quarter to the level of month.
- When drill-down is performed, one or more dimensions from the data cube are added.
- It navigates the data from less detailed data to highly detailed data.

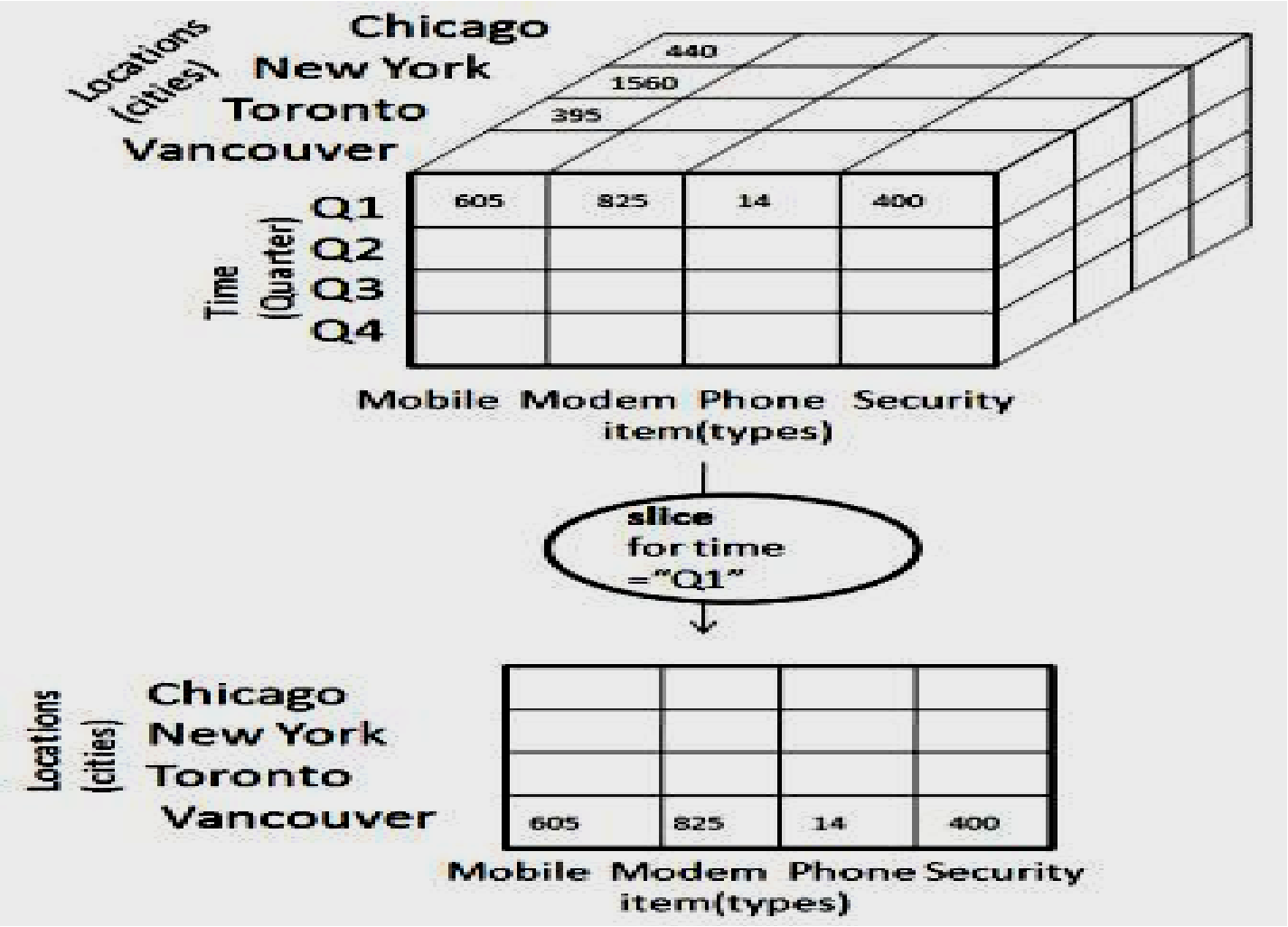
Diagram of how drill-down works



Slice

- The slice operation selects one particular dimension from a given cube and provides a new sub-cube.
  - ❖Here Slice is performed for the dimension "time" using the criterion time = "Q1".
  - ❖It will form a new sub-cube by selecting one or more dimensions.

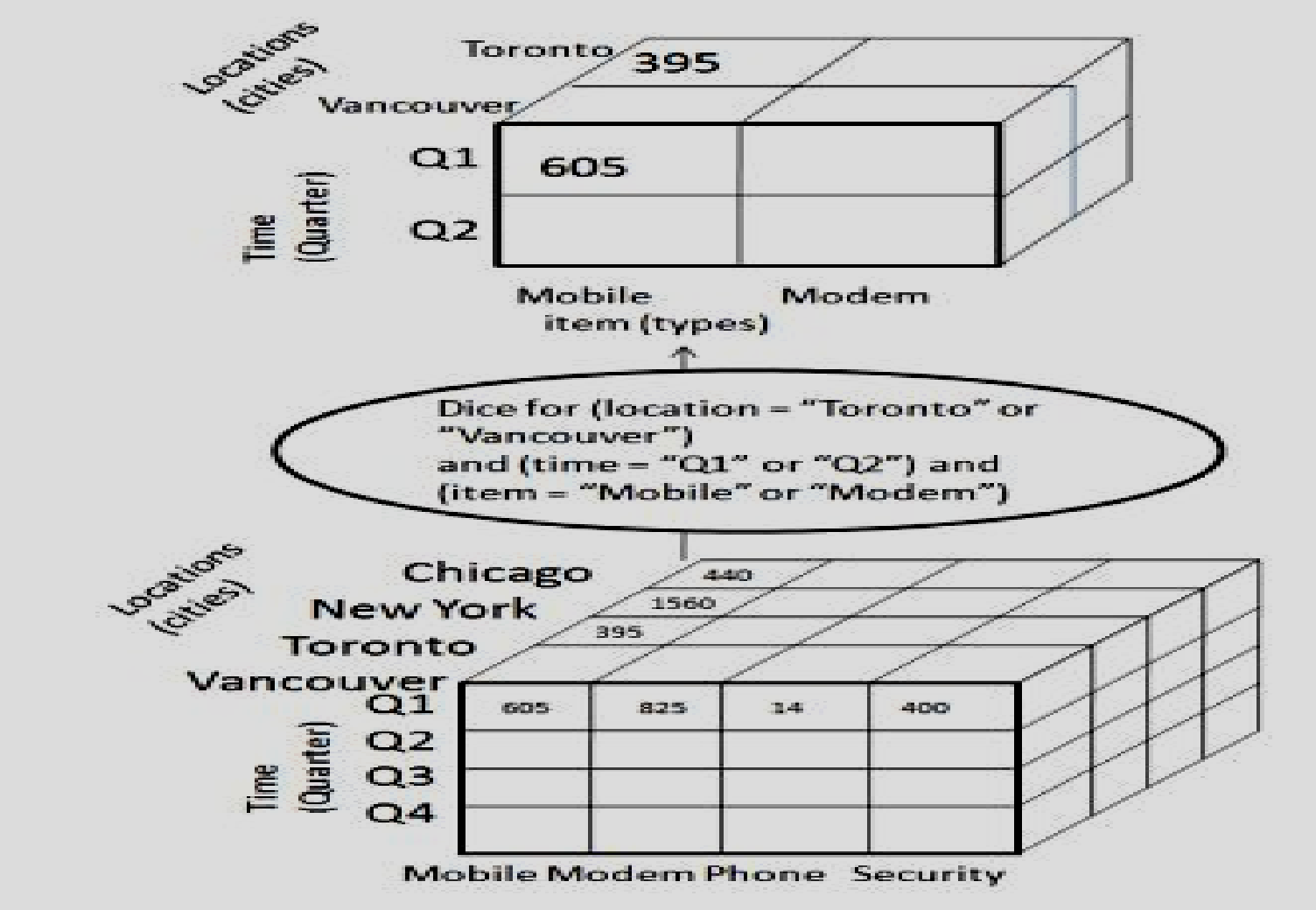
Diagram that shows how slice works.



Dice

- Dice selects two or more dimensions from a given cube and provides a new sub-cube.
- The dice operation on the cube based on the following selection criteria involves three dimensions.
  - ❖(location = "Toronto" or "Vancouver")
  - ❖(time = "Q1" or "Q2")
  - ❖(item =" Mobile" or "Modem")

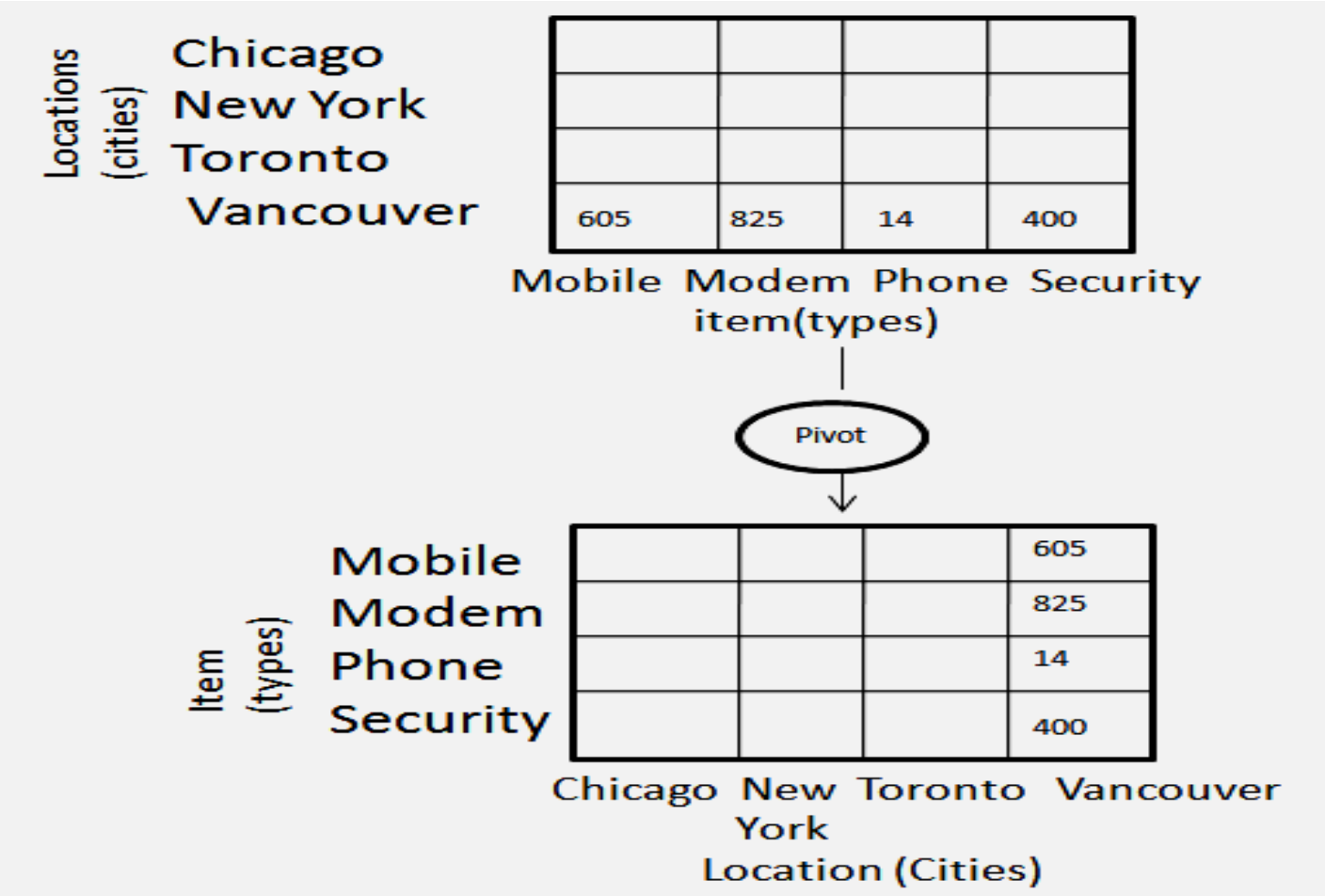
Diagram that shows the dice operation



Pivot

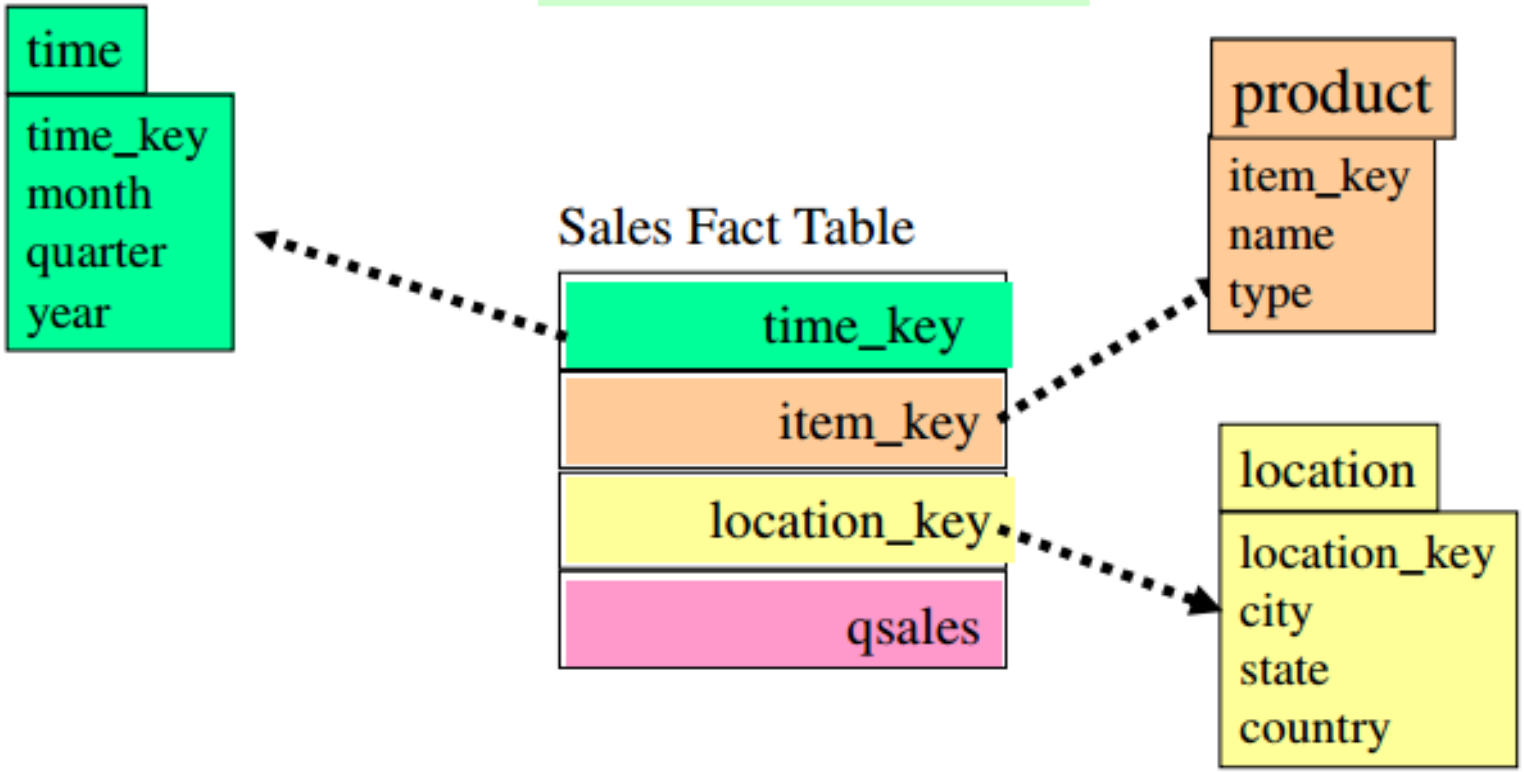
- The pivot operation is also known as rotation. It rotates the data axes in view in order to provide an alternative presentation of data.
- In this the item and location axes in 2-D slice are rotated.

Diagram that shows the pivot operation



OLAP operations and SQL

Simple star schema



Creation of tables

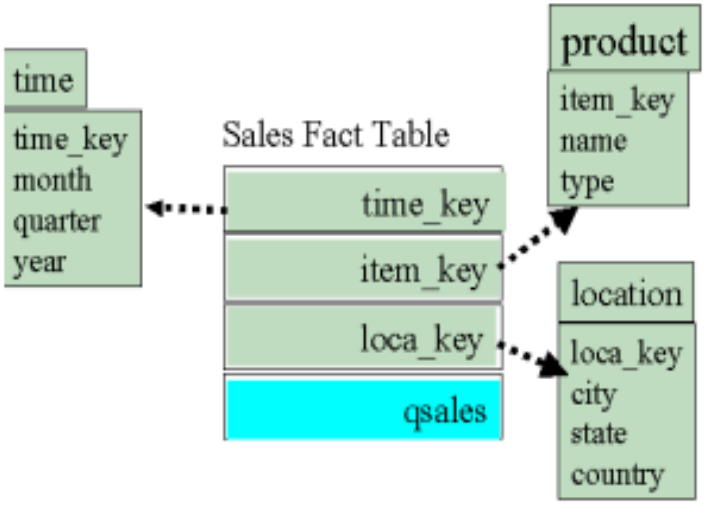
- create materialized view product as
  select name, type
  from 'DB1'.product

- create materialized view location as
  select city, state, country
  from 'DB2'.location

- The time dimension can be created in the same manner

Roll up query

month➔quarter




```
select    quarter, year, item_key,
          location_key, sum(qsales)
 from     sales S, time T
 where    S.time_key = T.time_key
group by  year, quarter, item_key, location_key
```

- In order to provide three dimensions we have to concatenate quarter and year in one string.

Drill down query

- Consider that roll up query is created as follows

```
create view salesQuarter as
    select   quarter, year, item_key, location_key, sum(q_sales)
      from   sales S, time T
     where   S.time_key = T.time_key
  group by   year, quarter, item_key, location_key
```

- Drill down(quarter➔month)
  - Use of salesQuarter, sales (provides qSales) and time dimesnsion

Slice query

```
create view  sliceView as
  select   time_key, item_key, location_key, q_sales
    from   sales S
   where   S.time_key = 1000
order by   item_key, location_key
```

Dice query

```
select   time_key, item_key, location_key, q_sales
 from    sliceView
where    sliceView.location_key = 2000
```

Dont forget order by

Data Warehouse Back-End Tools and Utilities

- Data warehouse systems use back-end tools and utilities to populate and refresh their data (Figure: 3-tier architecture). These tools and utilities include the following functions:
- Data extraction
- Data cleaning
- Data transformation
- Load
- Refresh

Data Warehouse Back-End Tools and Utilities

- **Data extraction**, which typically gathers data from multiple, heterogeneous, and external sources
- **Data cleaning**, which detects errors in the data and rectifies them when possible
- **Data transformation**, which converts data from legacy or host format to warehouse format
- **Load,** which sorts, summarizes, consolidates, computes views, checks integrity, and builds indices and partitions
- **Refresh**, which propagates the updates from the data sources to the warehouse

Data Warehouse-Tuning

- A data warehouse keeps evolving and it is unpredictable what query the user is going to post in the future.
- Therefore it becomes more difficult to tune a data warehouse system.
- In **Data Warehouse-Tuning**, we will discuss how to tune the different aspects of a data warehouse such as **performance**, **data load**, **queries**, etc.

Performance Assessment

- Here is a list of objective measures of performance:
  - Average query response time
  - Scan rates
  - Time used per day query
  - Memory usage per process
  - I/O throughput rates

Points to remember while Performance Assessment

- It is necessary to specify the measures in service level agreement (SLA).
- It is of no use trying to tune response time, if they are already better than those required.
- It is essential to have realistic expectations while making performance assessment.
- It is also essential that the users have feasible expectations.
- To hide the complexity of the system from the user, aggregations and views should be used.
- It is also possible that the user can write a query you had not tuned for.

Data Load Tuning

- Data load is a critical part of overnight processing. Nothing else can run until data load is complete. This is the entry point into the system.
- **Note**: If there is a delay in transferring the data, or in arrival of data then the entire system is affected badly. Therefore it is very important to tune the data load first.
- There are various approaches of tuning data load that are discussed below:
  - The very common approach is to insert data using the **SQL Layer**. In this approach, normal checks and constraints need to be performed. When the data is inserted into the table, the code will run to check for enough space to insert the data. If sufficient space is not available, then more space may have to be allocated to these tables. These checks take time to perform and are costly to CPU.

Data Load Tuning

- The second approach is to bypass all these checks and constraints and place the data directly into the preformatted blocks. These blocks are later written to the database. It is faster than the first approach, but it can work only with whole blocks of data. This can lead to some space wastage.
- The third approach is that while loading the data into the table that already contains the table, we can maintain indexes.
- The fourth approach says that to load the data in tables that already contain data, **drop the indexes & recreate them** when the data load is complete. The choice between the third and the fourth approach depends on how much data is already loaded and how many indexes need to be rebuilt.

Data Load Tuning

Integrity Checks

- Integrity checking highly affects the performance of the load. Following are the points to remember.
  - Integrity checks need to be limited because they require heavy processing power.
  - Integrity checks should be applied on the source system to avoid performance degrade of data load.

Tuning Queries

- We have two kinds of queries in data warehouse:
  - Fixed queries
  - Ad hoc queries

Fixed Queries

- Fixed queries are well defined. Following are the examples of fixed queries:
  - regular reports
  - Canned queries
  - Common aggregations
- Tuning the fixed queries in a data warehouse is same as in a relational database system. The only difference is that the amount of data to be queried may be different. It is good to store the most successful execution plan while testing fixed queries. Storing these executing plan will allow us to spot changing data size and data skew, as it will cause the execution plan to change.
- **Note**: We cannot do more on fact table but while dealing with dimension tables or the aggregations, the usual collection of SQL tweaking, storage mechanism, and access methods can be used to tune these queries.

Ad hoc Queries

- To understand **ad hoc queries**, it is important to know the ad hoc users of the data warehouse. For each user or group of users, you need to know the following:
  - The number of users in the group
  - Whether they use ad hoc queries at regular intervals of time
  - Whether they use ad hoc queries frequently
  - Whether they use ad hoc queries occasionally at unknown intervals.
  - The maximum size of query they tend to run
  - The average size of query they tend to run
  - Whether they require drill-down access to the base data
  - The elapsed login time per day
  - The peak time of daily usage
  - The number of queries they run per peak hour

Points to Note

- It is important to track the user's profiles and identify the queries that are run on a regular basis.
- It is also important that the tuning performed does not affect the performance.
- Identify similar and ad hoc queries that are frequently run.
- If these queries are identified, then the database will change and new indexes can be added for those queries.
- If these queries are identified, then new aggregations can be created specifically for those queries that would result in their efficient execution.

Testing the Data Warehouse:

- Testing is very important for data warehouse systems to make them work correctly and efficiently. There are four basic levels of testing performed on a data warehouse:
  - **Unit testing**
  - **Integration testing**
  - **System testing**
  - **User acceptance test**

Unit Testing

- In unit testing, each component is separately tested.
- Each module, i.e., procedure, program, SQL Script, Unix shell is tested.
- This test is performed by the developer.

Integration Testing

- In integration testing, the various modules of the application are brought together and then tested against the number of inputs.
- It is performed to test whether the various components do well after integration.

System Testing

- In system testing, the whole data warehouse application is tested together.
- The purpose of system testing is to check whether the entire system works correctly together or not.
- System testing is performed by the testing team.
- Since the size of the whole data warehouse is very large, it is usually possible to perform minimal system testing before the test plan can be enacted.

User Acceptance test

- User acceptance testing, the users of the data warehouse conducts their own tests on system.
- It is also one of the complete testing mechanisms of data warehouse.

Test Schedule

- First of all, the test schedule is created in the process of developing the test plan.
- In this schedule, we predict the estimated time required for the testing of the entire data warehouse system.
- There are different methodologies available to create a test schedule, but none of them are perfect because the data warehouse is very complex and large. Also the data warehouse system is evolving in nature.
- One may face the following issues while creating a test schedule:
  - A simple problem may have a large size of query that can take a day or more to complete, i.e., the query does not complete in a desired time scale.
  - There may be hardware failures such as losing a disk or human errors such as accidentally deleting a table or overwriting a large table.

- **Note**: Due to the above-mentioned difficulties, it is recommended to always double the amount of time you would normally allow for testing.

Testing Backup Recovery

- Testing the backup recovery strategy is extremely important.
- The list of scenarios for which this testing is needed:
  - Media failure
  - Loss or damage of table space or data file
  - Loss or damage of redo log file
  - Loss or damage of control file
  - Instance failure
  - Loss or damage of archive file
  - Loss or damage of table
  - Failure during data failure

Testing Operational Environment

- There are a number of aspects that need to be tested. These aspects are listed below.
- **Security** - A separate security document is required for security testing. This document contains a list of disallowed operations and devising tests for each.
- **Scheduler** - Scheduling software is required to control the daily operations of a data warehouse. It needs to be tested during system testing. The scheduling software requires an interface with the data warehouse, which will need the scheduler to control overnight processing and the management of aggregations.
- **Disk Configuration.** - Disk configuration also needs to be tested to identify I/O bottlenecks. The test should be performed with multiple times with different settings.
- **Management Tools.** - It is required to test all the management tools during system testing. Here is the list of tools that need to be tested.
  - Event manager
  - System manager
  - Database manager
  - Configuration manager
  - Backup recovery manager

Testing the Database

- The database is tested in the following three ways:
- **Testing the database manager and monitoring tools -** To test the database manager and the monitoring tools, they should be used in the creation, running, and management of test database.
- **Testing database features -** Here is the list of features that we have to test:
  - Querying in parallel
  - Create index in parallel
  - Data load in parallel
- **Testing database performance -** Query execution plays a very important role in data warehouse performance measures. There are sets of fixed queries that need to be run regularly and they should be tested. To test ad hoc queries, one should go through the user requirement document and understand the business completely. Take time to test the most awkward queries that the business is likely to ask against different index and aggregation strategies.

Testing the Application

- All the managers should be integrated correctly and work in order to ensure that the end-to-end load, index, aggregate and queries work as per the expectations.
- Each function of each manager should work correctly
- It is also necessary to test the application over a period of time.
- Week end and month-end tasks should also be tested.

Logistic of the Test

- The aim of system test is to test all of the following areas.
  - Scheduling software
  - Day-to-day operational procedures
  - Backup recovery strategy
  - Management and scheduling tools
  - Overnight processing
  - Query performance
- **Note**: The most important point is to test the scalability. Failure to do so will leave us a system design that does not work when the system grows.

Thank you !!!